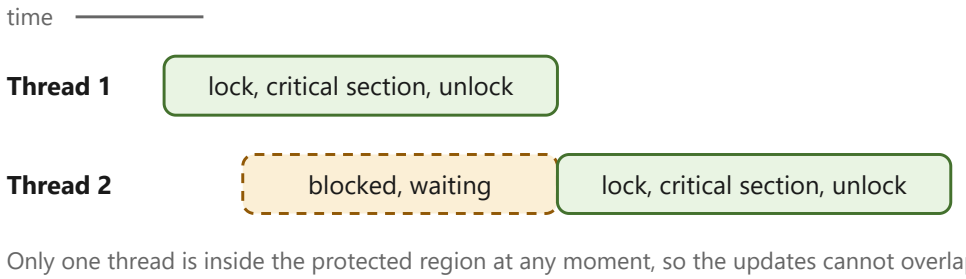
Mutexes and Locking

How a mutex makes threads take turns: RAII locks, protecting a class, the other mutex types, `call_once` and the races that hide in interfaces. Assumes Shared Data and Data Races.

Mutexes: taking turns



Only one thread is inside the protected region at any moment, so the updates cannot overlap.

Cost: thread 2 does nothing while it waits. Keep critical sections short.

A **mutex** lets one thread at a time into the **critical section**; others **wait** at `lock()`. Same two threads, now locked:

Run 500 random schedules: **always 4**.

```
std::mutex m;
int counter = 0;

void work() {
    for (int i = 0; i < 1000; ++i) {
        // locks now, unlocks automatically at the end of the block
        std::lock_guard<std::mutex> lock(m);
        ++counter;
    }
}
```

Use RAII locks, never raw `lock()` / `unlock()`

An exception or early return between `lock()` and `unlock()` leaves the mutex locked forever.

TYPE	USE IT WHEN
`std::lock_guard<std::mutex>`	default: lock for the whole scope
`std::unique_lock<std::mutex>`	unlock early, relock, defer; required by `cv.wait`
`std::scoped_lock` (C++17)	several mutexes at once, deadlock-free
`std::shared_mutex` + `shared_lock`	many readers or one writer

Protecting a class

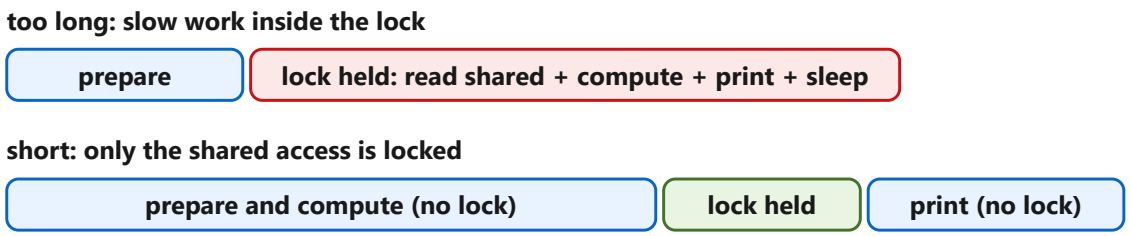
```
class SafeCounter {
    // mutable: so const getters can still lock it
    mutable std::mutex m_;
    int value_ = 0;
public:
    void increment()   { std::lock_guard<std::mutex> lk(m_); ++value_; }
    int  get() const   { std::lock_guard<std::mutex> lk(m_); return value_; }
};
```

GOOD LOCKING

- Keep critical sections **short**; do slow work (I/O, computation) outside.
- Mutex **next to its data**; never hand out references to that data.
- Never call unknown code (callbacks, virtuals) while locked.
- Too much locking means **contention**: the program becomes serial.

How long to hold a lock

A mutex serializes the code under it, so every extra line under the lock is time other threads spend waiting. The pattern is: do the slow parts first or after, and keep the lock for the shared access only.



Hold a lock for as short a time as possible, and never while doing slow work.

```
int total;                          // shared
std::mutex m;

int part = expensiveWork();         // no lock: private data
{
    std::lock_guard<std::mutex> lk(m);
    total += part;                  // the only shared access: lock it
}                                   // unlocked here
log(part);                          // slow output, outside the lock
```

DO UNDER THE LOCK	DO NOT DO UNDER THE LOCK
read or write the shared data	I/O, printing, sleeping
restore the invariant before unlocking	calling unknown code or callbacks
copy out what you need	waiting for another lock or a future

Other mutex types and `call_once`

TYPE	WHAT IT ADDS	USE WHEN
`std::mutex`	nothing: one owner, not re-lockable	the default
`std::recursive_mutex`	the **same thread** may lock it again; it must unlock as many times	almost never: see below
`std::timed_mutex`	`try_lock_for(d)` and `try_lock_until(t)`: give up after a time	a thread must not wait forever (watchdogs, real-time deadlines)
`std::shared_mutex` (C++17)	**shared** mode for readers, **exclusive** mode for writers	reads dominate and the critical section is not tiny

`recursive_mutex` is a design smell

```
class Account {
    std::recursive_mutex m_;
    int balance_ = 0;
public:
    void deposit(int n)      { std::lock_guard lk(m_); balance_ += n; }
    // locks the SAME mutex again
    void depositTwice(int n) { std::lock_guard lk(m_); deposit(n);
deposit(n); }
};
```

It works (compiled: balance 10), but it hides a design problem: `depositTwice` does not know what state `deposit` assumes. Prefer a **public** method that locks once and a **private** `depositLocked()` that assumes the lock is held. A plain `std::mutex` locked twice by one thread is undefined behaviour (usually a deadlock).

Reader-writer lock

```
class PriceTable {
    mutable std::shared_mutex m_;
    std::map<std::string, double> px_;
public:
    // exclusive: one writer, no readers
    void set(const std::string& k, double v) { std::unique_lock lk(m_);
px_[k] = v; }
    double get(const std::string& k) const {
        // shared: any number of readers together
        std::shared_lock lk(m_);
        auto it = px_.find(k); return it == px_.end() ? -1 : it->second;
    }
};
```

- A `shared_mutex` costs more than a `mutex` per lock. It pays off only when there are **many readers** and each read holds the lock for a while.
- Writers can starve if readers never leave (implementation-defined fairness).
- `try_lock()` returns immediately with true or false: compiled, a second thread's `try_lock` on a held mutex returned false; `try_lock_for(20 ms)` on one held for 200 ms returned false after 20 ms.

`std::call_once`

```
std::once_flag flag;
// eight threads may call init(): loadConfig runs exactly once
void init() { std::call_once(flag, [] { loadConfig(); }); }
```

Use it (or a function-local `static`) for lazy initialization shared by several threads, instead of a hand-written double-checked lock.

> KEY POINT "When would you use a recursive mutex?" Rarely, and say so: it usually means a method that locks calls another that also locks. Restructure into a locking public method and a lock-assuming private one.

Races hidden in interfaces

A mutex in every member function is necessary but **not sufficient**. Two threads run this on a shared stack whose methods all lock internally:

```
// shared by several threads; push, pop, top, empty all lock internally
SafeStack<int> s;

if (!s.empty()) {                  // 1. is there anything to take?
    int v = s.top();               // 2. read the top element
    s.pop();                       // 3. remove it
    use(v);
}
```

Each call is atomic, the **sequence is not**. Play A and B on a one-element stack:

Two failures, neither preventable by internal locking:

- B calls `top()` on an empty stack: undefined behavior (the `empty()` answer went stale).
- Both read `top()` before either pops: one element processed **twice**.

This **check-then-act** race also hits lock-free structures. The cause is the interface, so change the interface.

Fix: offer whole operations

```
template <typename T>
class SafeStack {
    std::stack<T> s_;
    mutable std::mutex m_;
public:
    void push(T v) {
        std::lock_guard<std::mutex> lk(m_);
        s_.push(std::move(v));
    }
    // check + read + remove as ONE atomic operation
    std::optional<T> try_pop() {
        std::lock_guard<std::mutex> lk(m_);
        if (s_.empty()) return std::nullopt;
        T v = std::move(s_.top());
        s_.pop();
```

```
        return v;
    }
        // empty() may still exist, but treat its answer as a hint only: it
    // can be stale the moment it returns.
};

// no window between the check and the removal
if (auto v = s.try_pop()) use(*v);
```

WHY DOES STD::STACK SPLIT TOP() AND POP()? If `pop()` returned by value and the copy threw after removal, the element would be lost. A concurrent version must merge them: out-parameter, `std::optional`, `shared_ptr<T>`, or a non-throwing move.

Same pattern elsewhere:

RACY SEQUENCE	WHOLE-OPERATION FIX
`if (!m.contains(k)) m[k] = v;`	one `insert` / `try_emplace` call that does both under the lock
`if (q.size() < max) q.push(x);`	a `try_push` that checks and pushes under one lock
`if (!file_exists(p)) create(p);`	an atomic "create if absent" call (for example the OS `O_EXCL` flag)
`if (counter.load() > 0) counter--;`	a `compare_exchange` loop

KEY POINT "Mutex in every method, so thread-safe?" No: each call is safe, the caller's *sequence* is not. Provide combined operations like `try_pop` and protect the **whole invariant**.

Further reading by section

Where each section of this lesson is explained in more depth. Pages are PDF viewer pages (the number your PDF reader shows).

SECTION	BOOK	PART OF THE BOOK	PDF PAGES
Mutexes: taking turns	Effective Modern C++	Item 16, Make const member functions thread safe	121 to 127
	C++ Concurrency in Action	3.2.1 to 3.2.3 Mutexes in C++, structuring code to protect data, race conditions inherent in interfaces	64 to 74
	C++ Concurrency in Action	3.2.6 to 3.2.8 std::unique_lock, transferring mutex ownership, locking granularity	82 to 87
Other mutex types and call_once	C++ Concurrency in Action	3.3.1 Protecting shared data during initialization (std::call_once)	88 to 91
Races hidden in interfaces	C++ Concurrency in Action	3.2.1 to 3.2.3 Mutexes in C++, structuring code to protect data, race conditions inherent in interfaces	64 to 74
	C++ Concurrency in Action	6.1 to 6.2.2 Designing for concurrency, a thread-safe stack and a queue with condition variables	197 to 206